Servicio Nacional de Aprendizaje

Análisis y Desarrollo de Software
Ficha: 3235872

Evidencia de producto:

Módulos integrados.

GA8-220501096-AA1-EV02.

Elaborado por:
Emmanuel Porras.

Centro de Servicios de Salud
Regional Antioquia

Octubre de 2026.

1. Introducción

Este documento entrega los módulos integrados del software web de la ESPM. Explica qué hace cada módulo, qué datos recibe y qué datos devuelve, cómo se configura el servidor y la base de datos, cómo están organizados los ambientes de desarrollo y pruebas, y cómo se instala, se ejecuta y se despliega el sistema.

El sistema tiene tres módulos que trabajan juntos: pago de facturas, quejas (PQR) y administración. Se construyó con JavaScript (Node.js), el framework Express y la base de datos SQLite. El diseño de las capas y los diagramas se explican con más detalle en el documento de la evidencia AA1-EV01.

2. Contenido de la entrega

La guía de la evidencia pide varios elementos. La siguiente tabla indica dónde encontrar cada uno.

Lo que pide la guía	Dónde está
Acta de aprobación de los requerimientos	Documento de actas, Acta 1
Repositorio de control de versiones	Sección 12 y enlace de GitHub
Archivos ejecutables	iniciar.bat y ejecutar-pruebas.bat en la carpeta del proyecto
URL donde se desplegaron los módulos	Sección 10
Documentación por módulo y componente, con datos de entrada y salida	Secciones 5 y 6
Informe de las pruebas de cada módulo y su resultado	Sección 11 y Acta 2
Configuración de servidores y de bases de datos	Sección 7
Ambientes de desarrollo y pruebas	Sección 8
Manual técnico	Sección 9
Acta de pruebas y aceptación	Documento de actas, Acta 2

3. Requerimientos del sistema

El sistema atiende a dos tipos de usuario. El usuario de la empresa consulta y paga sus facturas y radica quejas. El administrador crea facturas, responde quejas y revisa un resumen general.

Código	Requerimiento	Módulo
RF1	Consultar las facturas de una cuenta y el total pendiente	Pago de facturas
RF2	Pagar una factura pendiente con PSE, tarjeta o efectivo	Pago de facturas
RF3	Generar un recibo con referencia única y consultar el historial de pagos	Pago de facturas
RF4	Radicar una petición, queja o reclamo y recibir un número de radicado	Quejas
RF5	Consultar el estado y la respuesta de una queja	Quejas
RF6	Crear facturas nuevas	Administración
RF7	Listar todas las facturas y quejas, con filtro por estado	Administración
RF8	Responder quejas y cambiar su estado	Administración
RF9	Ver el resumen de dinero recaudado, pendiente y quejas por estado	Administración
RF10	Proteger las funciones del administrador con una clave	Seguridad

Los requerimientos no funcionales son: funcionar en un navegador web, guardar los datos sin instalar un servidor de base de datos aparte, validar todos los datos que escribe el usuario y mantener el código en un repositorio con control de versiones. El pago de facturas es una simulación: el sistema registra el pago y genera el recibo, pero no se conecta a un banco ni a una pasarela de pagos real.

4. Arquitectura

Las pantallas del navegador envían peticiones HTTP al servidor Express. El servidor revisa la clave si la ruta es del administrador, el módulo correspondiente aplica las validaciones y guarda o consulta los datos en SQLite. El módulo de administración reutiliza dos funciones de los otros módulos, por eso los tres trabajan integrados.

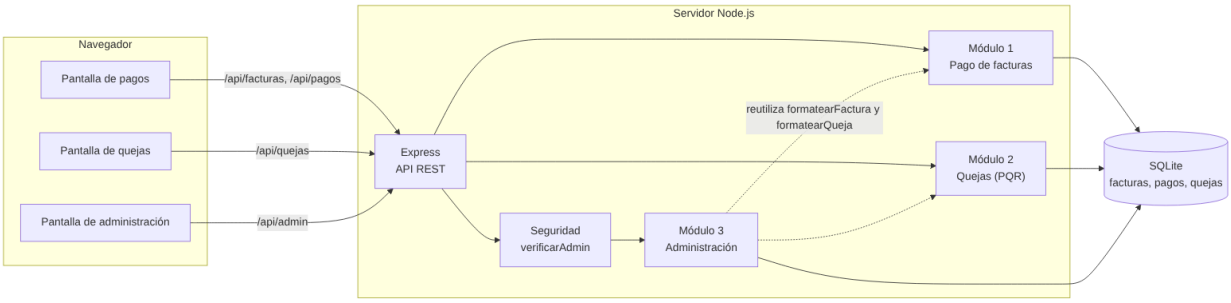


Figura 1. Componentes del sistema

5. Documentación por módulo

Cada servicio se describe con sus datos de entrada, sus datos de salida y los errores que puede devolver. Todas las respuestas son en formato JSON.

5.1 Módulo 1: pago de facturas

Archivos: src/routes/pagosRoutes.js y src/controllers/pagosController.js. Pantalla: public/index.html y public/app.js.

Servicio	Entrada	Salida	Errores
GET /api/facturas	Parámetro cuenta (obligatorio). Parámetro estado (opcional): PENDIENTE o PAGADA.	Número de cuenta, total pendiente y lista de facturas.	400: falta la cuenta o el estado no es válido. 404: la cuenta no tiene facturas.
POST /api/facturas/:id/pagar	El id de la factura en la ruta. Cuerpo: valor y metodoPago (PSE, TARJETA o EFECTIVO).	201: mensaje y recibo del pago.	400: id, método o valor incorrectos. 404: la factura no existe. 409: ya estaba pagada.
GET /api/pagos/:id	El id del pago en la ruta.	El recibo del pago.	400: id no válido. 404: el pago no existe.
GET /api/pagos	Parámetro cuenta (obligatorio).	Número de cuenta y lista de pagos, el más reciente primero.	400: falta la cuenta.

Ejemplo de consulta de facturas. Se muestra solo una de las tres facturas de la cuenta 1001.

```
Entrada:  GET /api/facturas?cuenta=1001
```

Salida (código 200):

```
{
  "numeroCuenta": "1001",
  "totalPendiente": 87600,
  "facturas": [
    {
      "id": 1,
      "numeroCuenta": "1001",
      "titular": "María Fernanda Rojas",
      "servicio": "Acueducto",
      "periodo": "2026-09",
      "valor": 48500,
      "fechaVencimiento": "2026-10-10",
      "estado": "PENDIENTE"
    }
  ]
}
```

Ejemplo de pago de una factura.

Entrada: POST /api/facturas/1/pagar con este cuerpo

```
{
  "valor": 48500,
  "metodoPago": "PSE"
}
```

Salida (código 201):

```
{
  "mensaje": "Pago registrado correctamente.",
  "recibo": {
    "idPago": 1,
    "referencia": "ESPM-20261029-000001",
    "fechaPago": "2026-10-29T21:58:12.026Z",
    "metodoPago": "PSE",
    "valorPagado": 48500,
    "factura": {
      "id": 1,
      "numeroCuenta": "1001",
      "titular": "María Fernanda Rojas",
      "servicio": "Acueducto",
      "periodo": "2026-09"
    }
  }
}
```

Salida con valor incorrecto (código 400):

```
{
  "error": "El valor a pagar debe ser exactamente $48500."
}
```

Salida si la factura ya estaba pagada (código 409):

```
{
  "error": "La factura ya fue pagada."
}
```

5.2 Módulo 2: quejas, peticiones y reclamos (PQR)

Archivos: src/routes/quejasRoutes.js y src/controllers/quejasController.js. Pantalla: public/quejas.html y public/quejas.js.

Servicio	Entrada	Salida	Errores
POST /api/quejas	Cuerpo: numeroCuenta, tipo (PETICION, QUEJA o RECLAMO), asunto (de 5 a 100 caracteres) y descripcion (de 10 a 1000 caracteres).	201: mensaje y la queja con su radicado y estado RADICADA.	400: algún dato no es válido. 404: la cuenta no existe.
GET /api/quejas	Parámetro cuenta (obligatorio).	Número de cuenta y lista de quejas, la más reciente primero.	400: falta la cuenta.
GET /api/quejas/:radicado	El radicado en la ruta, por ejemplo PQR-2026-000001.	La queja con su estado y respuesta.	404: no existe ese radicado.

Entrada: POST /api/quejas con este cuerpo

```
{
  "numeroCuenta": "1001",
  "tipo": "QUEJA",
  "asunto": "Cobro incorrecto",
  "descripcion": "Me cobraron el aseo dos veces."
}
```

Salida (código 201):

```
{
  "mensaje": "Queja radicada correctamente.",
  "queja": {
    "id": 1,
    "radicado": "PQR-2026-000001",
    "numeroCuenta": "1001",
    "tipo": "QUEJA",
    "asunto": "Cobro incorrecto",
    "descripcion": "Me cobraron el aseo dos veces.",
    "estado": "RADICADA",
    "respuesta": null,
    "fechaRadicacion": "2026-10-29T21:58:12.037Z",
    "fechaRespuesta": null
  }
}
```

Salida con un asunto de solo 4 caracteres (código 400):

```
{
  "error": "El asunto debe tener entre 5 y 100 caracteres."
}
```

5.3 Módulo 3: administración

Archivos: src/routes/adminRoutes.js, src/controllers/adminController.js y src/middlewares/verificarAdmin.js. Pantalla: public/admin.html y public/admin.js. Todos los servicios exigen el encabezado x-clave-admin; sin la clave correcta responden 401.

Servicio	Entrada	Salida	Errores
GET /api/admin/resumen	Solo el encabezado con la clave.	Dinero recaudado y pendiente, facturas pagadas y pendientes, y quejas por estado.	401: clave incorrecta o ausente.
GET /api/admin/facturas	Parámetro estado (opcional): PENDIENTE o PAGADA.	Total y lista de todas las facturas.	400: estado no válido.
POST /api/admin/facturas	Cuerpo: numeroCuenta, titular, servicio (Acueducto, Alcantarillado o Aseo), periodo (AAAA-MM), valor (entero mayor que cero) y fechaVencimiento (AAAA-MM-DD).	201: mensaje y la factura creada con estado PENDIENTE.	400: algún dato no es válido. 409: la factura ya existe para esa cuenta, servicio y periodo.
GET /api/admin/quejas	Parámetro estado (opcional): RADICADA, EN_TRAMITE o RESUELTA.	Total y lista de todas las quejas.	400: estado no válido.
PUT /api/admin/quejas/:id	El id en la ruta. Cuerpo: estado (EN_TRAMITE o RESUELTA) y respuesta (mínimo 5 caracteres si es RESUELTA).	200: mensaje y la queja actualizada.	400: id, estado o respuesta no válidos. 404: la queja no existe. 409: la queja ya estaba resuelta.

Entrada: POST /api/admin/facturas con este cuerpo

```
{
  "numeroCuenta": "1004",
  "titular": "Ana Torres",
  "servicio": "Acueducto",
  "periodo": "2026-10",
  "valor": 45000,
  "fechaVencimiento": "2026-11-10"
}
```

Salida (código 201):

```
{
  "mensaje": "Factura creada correctamente.",
  "factura": {
    "id": 7,
    "numeroCuenta": "1004",
    "titular": "Ana Torres",
    "servicio": "Acueducto",
    "periodo": "2026-10",
    "valor": 45000,
    "fechaVencimiento": "2026-11-10",
    "estado": "PENDIENTE"
  }
}
```

Salida con una fecha que no existe, como 2026-02-30 (código 400):

```
{
  "error": "La fecha de vencimiento debe tener el formato AAAA-MM-DD."
}
```

Entrada: PUT /api/admin/quejas/1 con este cuerpo

```
{
  "estado": "RESUELTA",
  "respuesta": "Se anuló el cobro repetido."
}
```

Salida (código 200):

```
{
  "mensaje": "Queja actualizada correctamente.",
  "queja": {
    "id": 1,
    "radicado": "PQR-2026-000001",
    "numeroCuenta": "1001",
    "tipo": "QUEJA",
    "asunto": "Cobro incorrecto",
    "descripcion": "Me cobraron el aseo dos veces.",
    "estado": "RESUELTA",
    "respuesta": "Se anuló el cobro repetido.",
    "fechaRadicacion": "2026-10-29T21:58:12.037Z",
    "fechaRespuesta": "2026-10-29T21:58:12.038Z"
  }
}
```

Salida de GET /api/admin/resumen (código 200):

```
{
  "recaudo": {
    "recaudado": 48500,
    "pendiente": 204200,
    "facturasPagadas": 1,
    "facturasPendientes": 6
  },
  "quejas": {
    "radicadas": 0,
    "enTramite": 0,
    "resueltas": 1
  }
}
```

Salida cuando falta la clave de administrador (código 401):

```
{
  "error": "Clave de administrador incorrecta o ausente."
}
```

6. Documentación de los componentes de apoyo

6.1 Base de datos

La base de datos SQLite tiene tres tablas. Se crean solas la primera vez que se enciende el servidor.

Tabla	Campos	Relación
facturas	id, numero_cuenta, titular, servicio, periodo, valor, fecha_vencimiento, estado (PENDIENTE o PAGADA)	Una factura puede tener cero o más pagos.
pagos	id, factura_id, valor_pagado, metodo_pago, referencia, fecha_pago	factura_id apunta a facturas.id.
quejas	id, radicado, numero_cuenta, tipo, asunto, descripcion, estado (RADICADA, EN_TRAMITE o RESUELTA), respuesta, fecha_radicacion, fecha_respuesta	Se relaciona con las facturas por numero_cuenta.

6.2 Componentes reutilizables

Estas piezas se escribieron una sola vez y las usan varios módulos.

Componente	Entrada	Salida
server.js	Variable de entorno PORT (opcional).	Servidor HTTP que entrega las pantallas de la carpeta public y la API en /api.
db.js	Ninguna: se carga al encender el servidor.	Conexión a la base de datos (prepare, exec, transaction). Crea las tablas y carga 6 facturas de ejemplo si está vacía.
verificarAdmin	Petición con el encabezado x-clave-admin.	Deja continuar la petición o responde 401.
formatearFactura(fila)	Fila de la tabla facturas, con guiones bajos.	Factura con nombres en camelCase.

Componente	Entrada	Salida
formatearQueja(fila)	Fila de la tabla quejas.	Queja con nombres en camelCase.
generarReferencia(idPago)	Número entero, el id del pago.	Texto como ESPM-20261029-000012.
generarRadicado(idQueja)	Número entero, el id de la queja.	Texto como PQR-2026-000007.
esFechaValida(texto)	Texto con una fecha.	true si tiene formato AAAA-MM-DD y la fecha existe; false si no.

7. Configuración de servidores y bases de datos

El servidor es Express y escucha en el puerto 3000. Si existe la variable de entorno PORT, usa ese puerto, que es lo que hacen los servicios de alojamiento. La base de datos es un archivo SQLite en data/espm.db que se crea automáticamente; no hay que instalar ni configurar ningún servidor de base de datos.

Variable	Valor por defecto	Para qué sirve
PORT	3000	Puerto donde escucha el servidor.
CLAVE_ADMIN	admin123	Clave del administrador. La de ejemplo solo debe usarse en pruebas; en el despliegue se debe cambiar.
NODE_ENV	No se define	Las pruebas lo cambian a test automáticamente para usar una base de datos temporal en memoria.

Para volver a empezar con datos nuevos se detiene el servidor, se borra la carpeta data y se enciende otra vez. La carpeta data no se sube al repositorio, porque está en el archivo. gitignore.

8. Ambientes de desarrollo y pruebas

Elemento	Desarrollo	Pruebas	Despliegue
Sistema operativo	Windows con PowerShell	El mismo	Servidor Linux del servicio de alojamiento
Node.js	Versión 24	Versión 24	La que use el servicio de alojamiento (mínimo 18)
IDE	Visual Studio Code	Terminal del IDE	No aplica
Base de datos	Archivo data/espm.db	SQLite en memoria	Archivo temporal; se pierde al reiniciar
Comando	npm.cmd start	npm.cmd test	npm install y npm start
Clave de administrador	admin123 o CLAVE_ADMIN	No se usa la clave real	CLAVE_ADMIN definida por el administrador
Datos	Se conservan entre ejecuciones	Se reinician antes de cada prueba	Se cargan las facturas de ejemplo al encender

Las librerías del proyecto son express y better-sqlite3. Las pruebas usan node:test y node:assert, que ya vienen con Node.js, así que no hay librerías extra de desarrollo.

9. Manual técnico

9.1 Requisitos previos

Se necesita Node.js versión 18 o superior (se recomienda la 24), Git y un navegador web. Para comprobar Node.js se abre una terminal y se escribe node -v.

9.2 Instalación

Primero se descarga el proyecto con git clone seguido del enlace del repositorio. Luego se abre una terminal dentro de la carpeta descargada y se escribe npm.cmd install, que descarga las librerías y crea la carpeta node_modules. En una terminal que no bloquee los scripts basta con npm install.

9.3 Ejecución

Se enciende el servidor con npm.cmd start, o haciendo doble clic en iniciar.bat, que además instala las librerías si faltan y abre el navegador. Cuando la terminal muestre el mensaje con http://localhost:3000, se abre esa dirección en el navegador.

Las cuentas de ejemplo son 1001, 1002 y 1003. La clave de prueba del administrador es admin123.

9.4 Pruebas

Las pruebas se ejecutan con npm.cmd test, o con doble clic en ejecutar-pruebas.bat. Al final se muestra cuántas pruebas pasaron y cuántas fallaron. Deben pasar las 101.

9.5 Estructura de carpetas

```
espm-pagos/
  server.js                 Enciende el servidor y conecta los módulos
  iniciar.bat               Enciende todo con doble clic (Windows)
  ejecutar-pruebas.bat      Ejecuta las pruebas con doble clic (Windows)
  package.json              Librerías y comandos del proyecto
  src/db.js                 Base de datos
  src/routes/               Rutas de cada módulo
  src/controllers/          Lógica y validaciones de cada módulo
  src/middlewares/          Revisión de la clave del administrador
  public/                   Pantallas (HTML, CSS y JavaScript)
  tests/                    Pruebas unitarias y de integración
  docs/                     Documentos y actas de la entrega
```

9.6 Cómo agregar un módulo nuevo

Primero se crea un archivo en src/controllers con las funciones del módulo; cada función recibe la petición (req) y la respuesta (res). Después se crea un archivo en src/routes que asocia cada dirección con su función. Luego se conecta ese archivo de rutas en server.js con app.use. Por

último, se crea un archivo en tests con terminación .test.js que pruebe cada función, y se ejecuta npm.cmd test para comprobar que todo pasa.

9.7 Solución de problemas

Qué se ve	Causa	Solución
PowerShell dice que la ejecución de scripts está deshabilitada	PowerShell bloquea el comando npm	Usar npm.cmd en lugar de npm
Error EADDRINUSE: address already in use :::3000	Otro programa, o el mismo servidor ya encendido, usa el puerto 3000	Cerrar el otro programa, o encender en otro puerto con $env:PORT=3001; npm.cmd start
Error de better-sqlite3 al instalar o encender	La versión de Node.js es muy antigua o cambió después de instalar	Instalar Node.js 18 o superior, borrar la carpeta node_modules y ejecutar npm.cmd install
Error 401: clave incorrecta	La clave escrita no coincide	Usar admin123 o la clave definida en CLAVE_ADMIN
Quiero empezar con datos nuevos	La base de datos guarda lo anterior	Detener el servidor, borrar la carpeta data y volver a encender
La página tarda en cargar en el sitio desplegado	El servicio gratuito se duerme cuando no tiene visitas	Esperar cerca de un minuto y recargar

10. Despliegue y URLs

Los módulos pueden desplegarse en un servicio de alojamiento para que cualquier persona los abra desde internet. Estos son los pasos en Render, que permite crear un servicio web gratuito conectado a GitHub.

Primero se entra a render.com y se inicia sesión con la cuenta de GitHub. Luego se elige New, después Web Service, y se selecciona el repositorio ESPM-modulos-. En el formulario se escribe un nombre, se deja la rama main, se escribe npm install como Build Command y npm start como Start Command, y se elige la instancia Free. En Advanced se agrega la variable de entorno CLAVE_ADMIN con una clave propia. Al terminar se pulsa Create Web Service y se espera a que el despliegue finalice. Render muestra la dirección pública, que termina en onrender.com.

El servicio gratuito tiene límites. Se duerme después de 15 minutos sin visitas y tarda cerca de un minuto en despertar. Además, el archivo de la base de datos se pierde cada vez que el servicio se reinicia o se despliega de nuevo, pero el sistema vuelve a cargar las seis facturas de ejemplo, así que sigue funcionando.

Recurso	URL
Repositorio en GitHub	https://github.com/Emmanuel-082/ESPM-modulos-
Aplicación desplegada (los tres módulos)	https://espm-modulos.onrender.com
Ejecución en el computador	http://localhost:3000

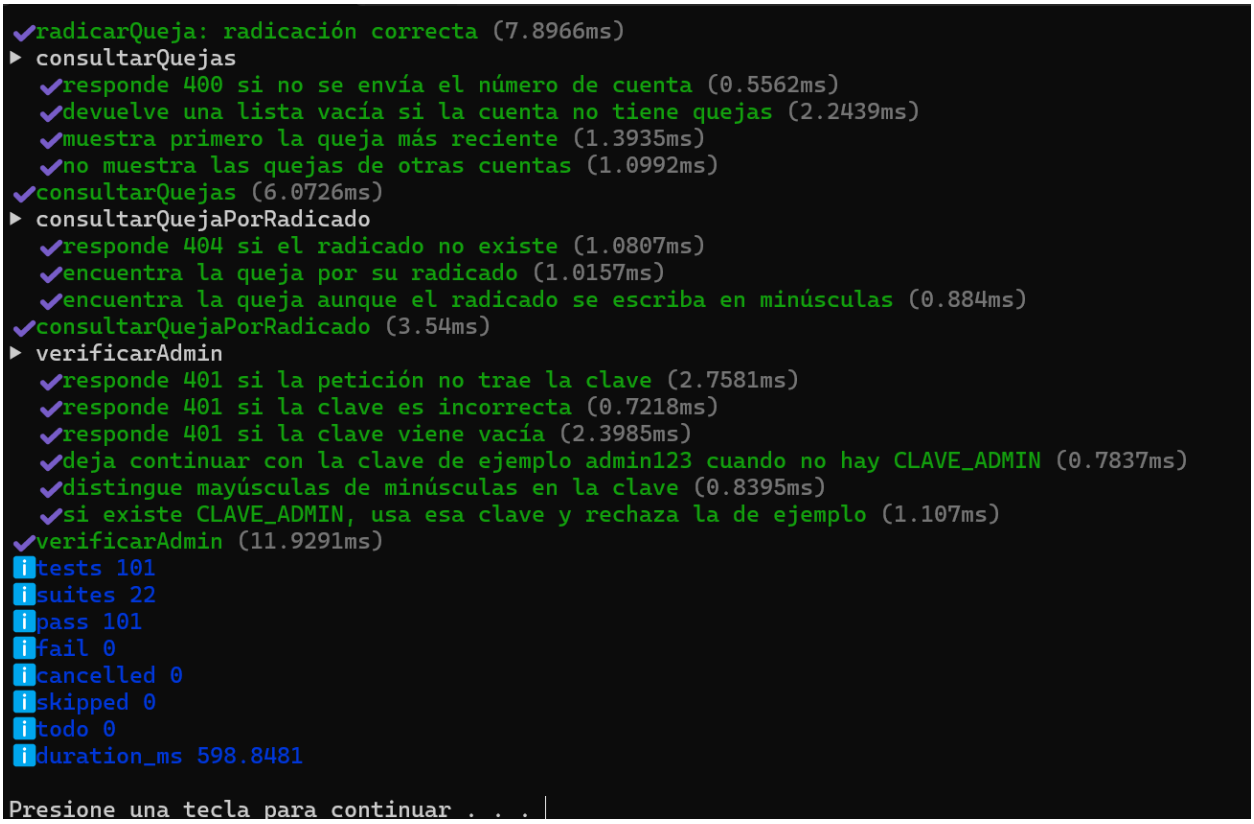
11.Informe de pruebas

Se aplicaron pruebas unitarias automáticas a cada módulo y a la seguridad, y pruebas de integración entre los módulos. Cada prueba llama a la función del controlador con datos de ejemplo y comprueba el código de respuesta y los datos devueltos. Se probaron los casos correctos y los casos de error: campos vacíos, textos muy cortos o muy largos, caracteres especiales, fechas que no existen, números negativos o decimales, valores no permitidos, registros inexistentes o repetidos, e intentos de inyección de SQL.

Módulo	**Archivo**	**Pruebas**	**Aprobadas**	**Resultado**
Pago de facturas	pagos.test.js	30	30	Aprobado
Quejas	quejas.test.js	26	26	Aprobado
Administración	admin.test.js	33	33	Aprobado
Seguridad	verificarAdmin.test.js	6	6	Aprobado
Integración	integracion.test.js	6	6	Aprobado
Total		**101**	**101**	**Aprobado**

Las pruebas de integración comprueban, por ejemplo, que una factura creada por el administrador aparece en la consulta del usuario, que un pago del usuario se refleja en el resumen del administrador y que una queja respondida por el administrador se ve en la consulta por radicado. El detalle de los casos aplicados y la aceptación están en el Acta 2.

Captura de la ejecución de las pruebas (pegue aquí la imagen de su terminal después de ejecutar npm.cmd test):



12. Control de versiones

El código fuente se maneja con Git y se publica en GitHub, en https://github.com/Emmanuel-082/ESPM-modulos-. Cada avance se guardó como un commit con un mensaje que explica el cambio. Los documentos de esta entrega están en la carpeta docs del repositorio. Las carpetas node_modules y data y el archivo .env no se suben, porque están en .gitignore.

13. Lista de chequeo

No.	Indicador de logro	Evidencia
1	Entrega los módulos codificados y documentados	Código en el repositorio con comentarios en cada función, y documentación por módulo en las secciones 5 y 6.
2	Entrega documento técnico del sistema	Este documento.
3	Entrega el ambiente de desarrollo y pruebas	Sección 8, package.json, iniciar.bat y ejecutar-pruebas.bat.
4	Entrega el código por medio del manejo de control de versiones	Repositorio de GitHub de la sección 12.
5	Entrega acta con aplicación de pruebas y aceptación	Documento de actas, Acta 2.